ELETRÔNICA DIGITAL

Maikon Lucian Lenz



SAGAH
SOLUÇÕES EDUCACIONAIS INTEGRADAS

Lógica sequencial

Objetivos de aprendizagem

Ao final deste texto, você deve apresentar os seguintes aprendizados:

- Conceituar flip-flops e registradores.
- Analisar circuitos síncronos e assíncronos.
- Utilizar máquinas de estados finitos.

Introdução

Métodos de sequenciamento de operação foram criados a partir do desenvolvimento de circuitos digitais capazes de armazenar, mesmo que temporariamente, um nível lógico. Os circuitos responsáveis por esse armazenamento são normalmente chamados de latch ou flip-flop, esse último contendo quatro tipos básicos de construção com formas de operação diferentes.

Neste capítulo, você vai estudar os flip-flops tipo RS, D, T e JK, bem como uma variação do JK mestre-escravo. A partir do estudo dos flip-flops, você vai verificar os circuitos síncronos e assíncronos, dentre os quais se destacam os contadores, de grande utilidade para a implementação de máquinas de estados. Trata-se de uma metodologia capaz de lidar de maneira sistemática com problemas complexos por meio do sequenciamento das operações.

Flip-flops e registradores

A saída de circuitos combinacionais depende exclusivamente das entradas em dado instante de tempo. Esse tipo de operação é capaz de resolver inúmeros problemas, mas é incapaz de solucionar qualquer situação que demande memória. Os **circuitos sequenciais** visam suprir a limitação operacional dos circuitos combinacionais, tornando a saída, em conjunto com as entradas, relevante para a nova transição de estado, conforme leciona Tocci (2011). Além da capacidade de memorização, os circuitos sequenciais envolvem temporização, para que sejam capazes de reagir a respeito da sua própria saída.

Nos circuitos combinacionais, o principal elemento é a **porta lógica**. O conjunto delas, cada qual com sua função, sendo conectadas de forma adequada, visa a solucionar um problema expresso por uma tabela verdade. Nos circuitos sequenciais, o elemento principal é o **multivibrador biestável**, normalmente chamado de flip-flop (FF), o qual é constituído por um conjunto de portas lógicas com retorno da saída para a entrada em algum ponto, conforme define Tokheim (2013).

Há, portanto, mais um nível de abstração. Os transistores estão ocultos dentro das portas lógicas, para as quais é estabelecido um comportamento rígido vinculado à uma tabela verdade, desde que obedecidas as configurações elétricas mínimas do fabricante. O flip-flop, no entanto, vai além, e o bloco que o representa oculta as próprias portas lógicas, sendo apresentados apenas os seus pinos de entrada e saída e, novamente, a tabela verdade para o comportamento esperado em cada situação.

São quatro tipos de flip-flops básicos: tipo RS, tipo JK, tipo D e tipo T. É claro que, dentro de cada categoria, há variações de comportamento e construção, mas o princípio básico de cada tipo é sempre o mesmo. Outra característica comum a todos eles são as saídas complementares. Todo flip-flop possui uma saída condizente com a lógica de funcionamento, denominada Q, e outra invertida, denominada $\overline{Q}$ (lê-se Q barra, Q barrado ou não Q), conforme leciona Tokheim (2013).

O flip-flop é derivado de um componente similar e um pouco mais simples, denominado latch. A diferença fundamental entre flip-flops e latches é que o primeiro possui mecanismos de controle das entradas — normalmente um pino de habilitação das entradas —, enquanto o segundo lida diretamente com as entradas e reage imediatamente a qualquer alteração.



Fique atento

Latch e flip-flop são facilmente confundidos. A diferença fundamental entre os dois termos é que o latch representa qualquer circuito lógico de armazenamento temporário de memória, enquanto os flip-flops podem ser mais complexos e possuem controle de suas entradas, para habilitar ou não a função de determinados pinos.

Latch e flip-flop tipo RS

Os latches e flip-flops do **tipo RS** — também chamados S-R, abreviação em inglês para SET-RESET — levam esse nome por conta da função exercida por

cada um dos pinos de entrada, sendo o **pino S** utilizado para inicializar e o **pino R** para reinicializar a saída. As entradas podem ser consideradas ativas em nível baixo ou alto, dependendo do modelo ou da forma como é construído o dispositivo, conforme leciona Floyd (2007).

Quando somente a entrada S estiver ativa, espera-se que o flip-flop mude — caso já não esteja alterado — o estado do pino de saída direta (Q) para 1. Quando somente a entrada R estiver ativa, é esperado que o flip-flop mude — caso já não esteja alterado — o estado do **pino de saída direta** (Q) para 0. Obviamente o pino $\overline{Q}$ (saída invertida ou complementar) fará o processo inverso.

Quando nenhuma das duas entradas estiverem ativas, o FF deverá manter o valor da saída no estado em que está; ou seja, se o último comando foi dado para mudar para 1, mantenha 1, caso contrário, mantenha em 0, conforme afirma Tokheim (2013). A última hipótese, em que ambos os pinos de entrada estejam ativos, é considerada uma condição inválida, já que as saídas, que deveriam ser complementares, serão ambas iguais a 1, devido ao circuito interno do latch/FF do tipo RS. Ao desativar as entradas, não será possível determinar exatamente o estado da saída, que dependerá de qual entrada saiu do estado ativo primeiro, segundo Tocci (2011).

Na Figura 1 são apresentados os FFs com entradas ativas em 1 (Figuras 1a e 1b) e em 0 (Figuras 1c e 1d).

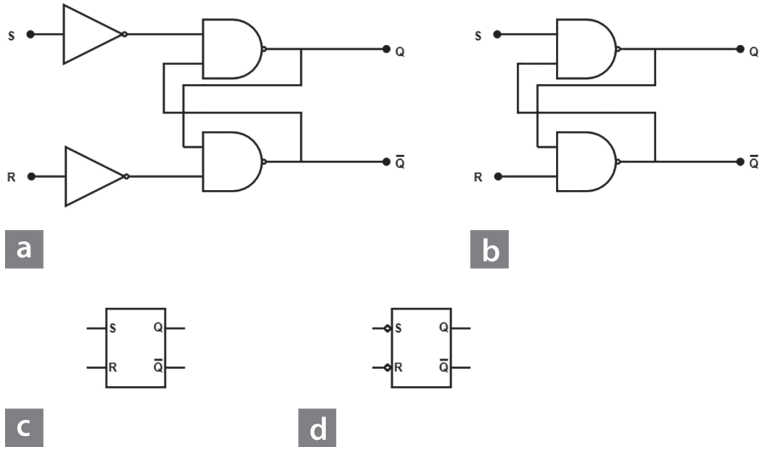


Figura 1. a) FF tipo RS com entradas ativas em 1; b) abstração do FF em (a); c) FF tipo RS com entradas ativas em 0; d) abstração do FF em (c).

À primeira vista, apesar de a lógica ser menos intuitiva, pode-se imaginar que o FF com entradas ativas em 0 seja mais difundido pela economia de portas lógicas. Entretanto, a maioria dos FFs utilizados possuem uma AND para cada entrada, em conjunto com um pino de habilitação (E, do inglês *enable*). Isso possibilita o uso de uma porta NAND para o FF de entradas ativas em 1, colocando ambas as soluções em mesma quantidade de portas lógicas utilizadas (Figura 2). Assim, é mais provável que a maior parte dos FF tipo RS operem com entradas ativas em 1, seja por facilitar a análise, seja devido à produção mais eficiente, derivada da utilização de apenas portas NAND na sua construção, quando da presença de pinos de habilitação.

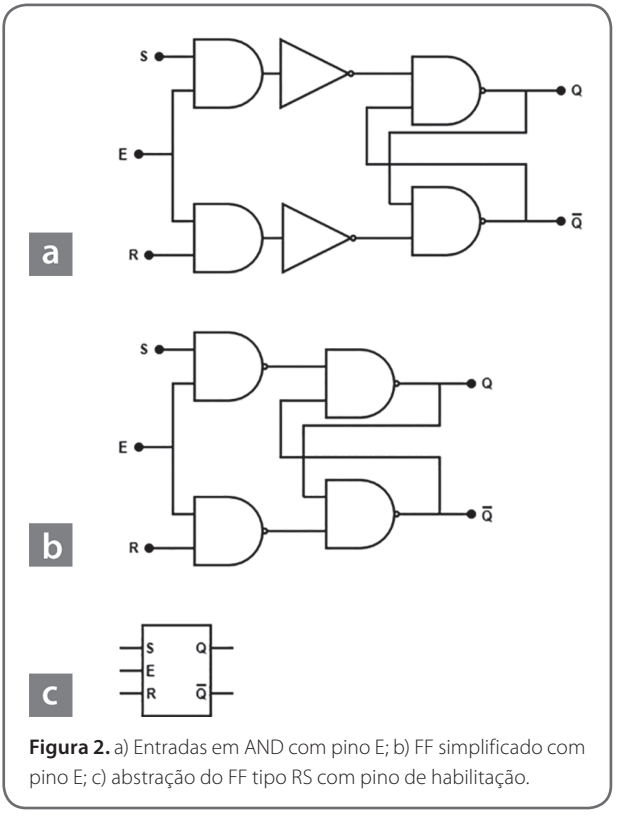


Figura 2. a) Entradas em AND com pino E; b) FF simplificado com pino E; c) abstração do FF tipo RS com pino de habilitação.

O FF tipo RS pode ser utilizado como uma memória, mas é pouco prático para essa função, porque necessita de duas entradas: o pino S para gravar o valor 1, e o pino R para apagar. A operação pode ser descrita pela tabela verdade mostrada no Quadro 1.

Quadro 1. Tabela verdade do flip-flop tipo RS

E	*S*	*R*	*Q*	$\bar{Q}$
0	0	0	*Q* (mantém)	$\bar{Q}$ (mantém)
0	0	1		
0	1	0		
0	1	1		
1	0	0		
1	0	1	0	1
1	1	0	1	0
1	1	1	∄ — Inválida	

Antes que o pino de habilitação seja conectado a uma AND ou NAND com as demais entradas, ele pode passar por um circuito detector de borda, normalmente composto por um **multivibrador monoestável não redisparável**. A função desse multivibrador é fornecer 1 na saída por um período fixo, ao ser detectado 1 na entrada. Isso evitará que o pino de habilitação seja alterado com uma frequência maior do que o tempo mínimo de propagação necessário para o bom funcionamento do FF. Também evitará que as portas AND ou NAND aceitem alterações em S ou R com frequências menores do que a estabelecida para o funcionamento adequado do circuito, e outros casos como a trepidação de chaves mecânicas, algumas vezes utilizadas para fornecer os níveis de entrada em partes dos circuitos lógicos. A trepidação forneceria, por um breve instante de tempo, valores 1 e 0, o que produziria mudanças falsas de entrada, conforme expõe Tocci (2011).

Latch e flip-flop tipo D

O flip-flop tipo D pode ser obtido utilizando um único pino que funcione como sinal direto para o pino S e o seu inverso para o tipo R (Figura 3). Com isso, a tabela verdade é reduzida, removendo-se uma entrada. Sem o pino de habilitação, o latch tipo D tem pouca serventia, uma vez que apenas repete a entrada na saída Q e o inverso na saída $\bar{Q}$. Porém, com o uso do pino de habilitação, assim como já ocorria com o tipo RS, as mudanças de estado na entrada só terão validade quando o pino de habilitação for 1, o que permitirá

ao FF tipo D funcionar como uma memória, ao contrário do tipo RS, que necessita de apenas um pino de entrada. Sempre que o pino E for 1, a saída vai copiar o valor presente no pino de entrada D (do qual derivam as ligações para S, e o inverso para R, como dito anteriormente).

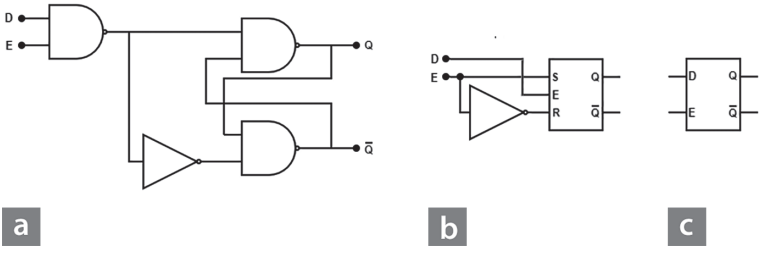


Figura 3. a) FF tipo D com portas NAND; b) bloco do FF tipo RS convertido para um tipo D; c) abstração do FF tipo D.

No Quadro 2, repare que a tabela verdade do FF tipo D não possui nenhuma condição inválida, como ocorria com o tipo RS. Isso porque, ao utilizar um único pino de entrada para S e o inverso para R, isso impossibilitará que ambos os pinos S e R sejam iguais a 1, como ocorria na condição inválida do tipo RS.

Quadro 2. Tabela verdade do flip-flop tipo D

E	*D*	*Q*	$\bar{Q}$
0	0	*Q* (mantém)	$\bar{Q}$ (mantém)
0	1		
1	0	0	1
1	1	1	0

Flip-flop tipo JK

O flip-flop tipo JK difere do FF RS apenas na condição em que todas as entradas estão ativas ao mesmo tempo. Para o FF RS, essa era uma condição funcionalmente inválida e poderia acarretar em resultados inesperados. O JK soluciona esse problema, inserindo a função de **mudança de estado** no lugar da condição inválida, conforme leciona Tokheim (2013).

Internamente, o FF JK é similar ao FF RS com pino de habilitação, porém, há um retorno a mais do que no tipo RS. As portas NANDs ligadas às entradas e ao pino E no tipo RS são substituídas por portas NANDs de três entradas, que recebem, além de um pino de função e habilitação, o retorno da saída oposta à que pretendem acionar (Figura 4). Ou seja, a saída direta Q alimenta a NAND dos pinos E e K, que são utilizados em conjunto para desligar a saída direta do FF; já a saída invertida $\overline{Q}$ alimenta a NAND dos pinos E e J, que são utilizados para ligar a saída direta do FF. Assim, por mais que ambas as entradas, J e K, sejam 1, somente a entrada de função oposta à atual situação do FF será habilitada. Em resumo, se $Q = 1$, somente K será validado, e o FF muda para $Q = 0$; mas, se $Q = 0$, então somente J será habilitado, e a saída mudará para $Q = 1$, conforme mostra o Quadro 3.

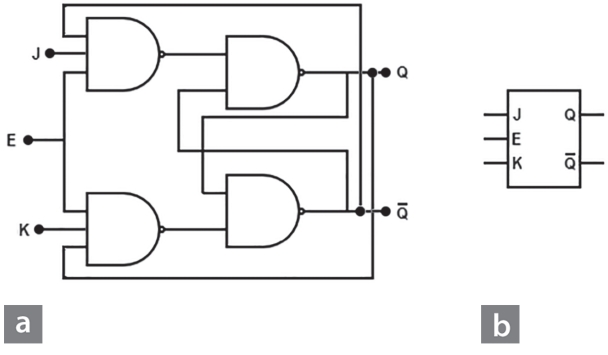


Figura 4. a) FF tipo JK; b) abstração do FF tipo JK.

Quadro 3. Tabela verdade do flip-flop tipo JK

E	*J*	*K*	*Q*	$\bar{Q}$
0	0	0	*Q* (mantém)	$\bar{Q}$ (mantém)
0	0	1		
0	1	0		
0	1	1		
1	0	0		
1	0	1	0	1
1	1	0	1	0
1	1	1	$\bar{Q}$ (inverte)	*Q* (inverte)

Entretanto, esse circuito exige que o circuito detector de borda seja extremamente preciso, curto o suficiente para que a saída não seja alterada duas vezes, mas longo o suficiente para que pelo menos uma mudança ocorra. Para reduzir ainda mais o risco, foi criado o FF JK mestre-escravo (Figura 5), correspondendo a dois FF RS. O segundo recebe as saídas do primeiro como entrada, enquanto o primeiro recebe os próprios pinos de entrada e um retorno das saídas do segundo FF. Mas a principal mudança está no pino de habilitação, já que cada bloco RS é acionado com sinais opostos no pino E. Enquanto o primeiro FF necessita que E seja 1 para funcionar, o segundo precisa que o pino E seja 0. Assim, a saída do primeiro FF prepara as entradas do segundo, que só vai acionar ao final de um ciclo completo de habilitação.

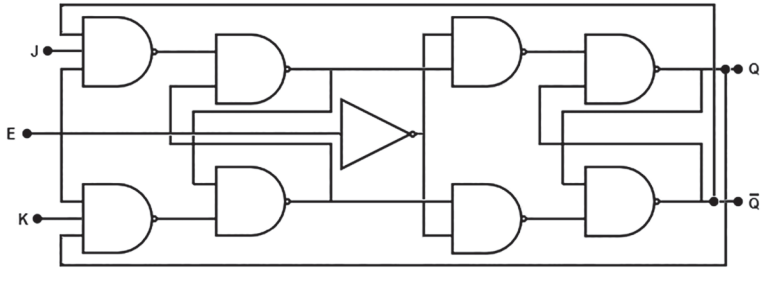


Figura 5. FF tipo JK mestre-escravo.

Flip-flop tipo T

No flip-flop tipo T, as entradas J e K do tipo JK são conectadas uma à outra (Figura 6) e, portanto, serão sempre iguais. Caso o pino E seja 1, as entradas estarão habilitadas e comandarão as saídas. Se o pino T for 0, J e K serão 0, e, portanto, não há mudança de estado; mas, se T for 1, então J e K serão 1, e haverá mudança de estado. O nome T deriva da sua **função de alternância** — em inglês, *toggle*.

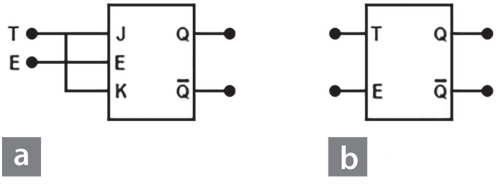


Figura 6. a) FF JK com as entradas J e K unidas em um único pino T; b) representação do FF tipo T.

A tabela verdade com todas as possibilidades para o FF tipo T é apresentada no Quadro 4.

Quadro 4. Tabela verdade do flip-flop tipo JK

E	*T*	*Q*	$\overline{Q}$
0	0	*Q* (mantém) 1	$\overline{Q}$ (mantém) 0
0	1		
0	0		
1	0		
1	1	$\overline{Q}$ (inverte)	*Q* (inverte)

Registradores

Os FFs são agrupados para representar números ou quantidades maiores; nesse caso, o conjunto deles passa a ser chamado de registrador. Enquanto cada um

dos FFs de um registrador só é capaz de armazenar um único bit, se considerados os valores de um grupo deles como sendo uma única variável, em que cada FF possui um peso diferente, pode-se obter dados de tamanhos maiores.

Os registradores normalmente são construídos em múltiplos de um byte (8 bits). São amplamente utilizados como *buffer* e para armazenamento temporário de dados. Também podem ser utilizados para deslocar dados. Nesse caso, os FFs estão ligados em série — e não em paralelo — e os dados fluem para o FF seguinte, criando um deslocamento dos bits.

Circuitos síncronos e assíncronos

Quando as entradas dos FFs dependem de um pino de controle, como é o caso do pino de habilitação, o FF é dito **síncrono**. Isso porque as entradas precisam estar literalmente sincronizadas com o pino de habilitação para que sejam funcionais. Muitas vezes, esse pino será utilizado por um **circuito temporizador**, que alterna entre períodos em nível alto e nível baixo de maneira periódica e com intervalo de tempo constante entre eles. Esse pulso regular e periódico é denominado ***clock*** e é utilizado pelos pinos de habilitação de cada FF que compõem o circuito, a fim de sincronizar a operação de todos eles. É muito comum que a representação do pino E seja substituída por um símbolo de > (Figura 7b), conforme leciona Tocci (2011).



Saiba mais

As entradas de um FF podem ser ativadas com nível baixo ou nível alto. Da mesma maneira, o pino de *clock* de um FF pode ser desenvolvido para atuar de uma forma ou de outra. Porém, no *clock*, o que importa é a transição do sinal, já que um circuito detector de borda é utilizado. Logo, diz-se que o *clock* pode ser acionado pela borda positiva (saindo de 0 para 1) ou pela borda negativa (saindo de 1 para 0).

Na ausência de uma entrada de controle, as funções são ditas **assíncronas** e podem operar a qualquer instante de tempo — basta que sejam acionadas. Há vantagens e desvantagens para cada tipo de operação e, principalmente, objetivos diferentes entre elas.

Mesmo FFs síncronos possuem, na maioria das vezes, entradas assíncronas para fins específicos. É comum FFs do tipo JK possuírem entradas PRESET (PS) e CLEAR (CLR), que têm a mesma função de J e K, respectivamente, mas que operam no circuito mais próximo da saída e, portanto, não dependem do pino de controle, podendo ligar ou desligar a saída em qualquer instante de tempo. Esses dois pinos são apresentados barrados, caso funcionem a partir de uma entrada igual a 0 ou caso funcionem a partir de entradas iguais a 1 (Figura 7). É o caso, por exemplo, de um contador, que incrementa o valor da saída a cada ciclo completo de *clock*, mas pode ser reiniciado a qualquer instante de tempo, utilizando um pino que não passa pela entrada de controle.

Imagine um cronômetro, que você pretende acionar ao dar a largada de uma corrida e parar exatamente ao término dela. O sistema será mais preciso caso as entradas que dão início e término à contagem sejam assíncronas, já que sua ordem será imediatamente interpretada pelo circuito. Mas, internamente, o restante das funções vai ocorrer de maneira síncrona, para que as mudanças de estado ocorram apenas no tempo exato. Nesse caso, ambos os métodos são empregados de forma correta. O cronômetro não funcionaria adequadamente ou teria seu desempenho prejudicado se a operação de contagem fosse assíncrona ou se a operação de comando fosse síncrona.

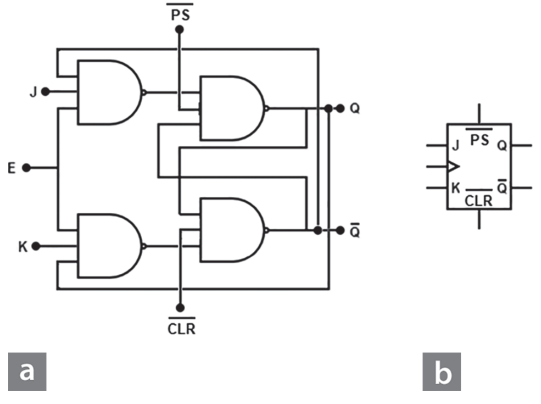


Figura 7. a) FF do tipo JK com entradas de PRESET e CLEAR; b) representação em bloco do FF JK com PRESET e CLEAR.

O uso desses circuitos com memória, mesmo que temporária, permitiu a implementação de **contadores**, que podem ser construídos de duas formas: síncrona ou assincronamente. Independentemente do tipo de contador, esses circuitos são amplamente utilizados, estando presentes na maioria dos processadores e outros circuitos digitais de funções complexas, conforme leciona Tokheim (2013).

Contadores assíncronos

Uma unidade binária só pode representar dois números em decimal: 0 ou 1. Para que um número maior seja representado, são necessários mais bits. A cada novo bit, o valor máximo em decimal dobra. Uma vez que cada FF também só pode representar um bit, é necessário um FF para cada bit adicional. Naturalmente, o circuito será menor se forem utilizados FFs que possuam apenas uma entrada para operar as saídas, como é o caso dos tipos D e T.

Um contador assíncrono pode ser construído utilizando-se FFs do tipo T com entrada permanente em nível 1, aproveitando que, para cada pulso de *clock*, o primeiro bit vai alternar os valores. Para que os demais bits não alternem a cada pulso de *clock*, a entrada de *clock* do FF seguinte recebe a saída do FF anterior. Isso fará com que o sinal de *clock* percebido por esse FF seja igual a dois ciclos completos do primeiro. Pode-se, então, aumentar a quantidade de bits, procedendo da mesma maneira para os próximos, sempre utilizando a saída do FF atual como entrada de *clock* do FF seguinte.

Cada FF precisa do dobro de tempo necessário se comparado ao anterior para alternar o estado de sua saída. Se considerado cada FF como sendo uma unidade binária de peso diferente, o resultado será incrementos unitários a cada pulso de *clock* de um número equivalente em decimal a 2^N, onde N corresponde à quantidade de FFs utilizados.

Todo o processo é demonstrado na Figura 8; porém, repare que não são utilizados FFs do tipo T, mas, sim, FFs do tipo JK. De qualquer forma, o funcionamento será o mesmo, já que ambas as entradas do FF JK estão sempre em nível 1, o que caracteriza a operação de alternância de um FF do tipo T com uma única entrada em nível 1.

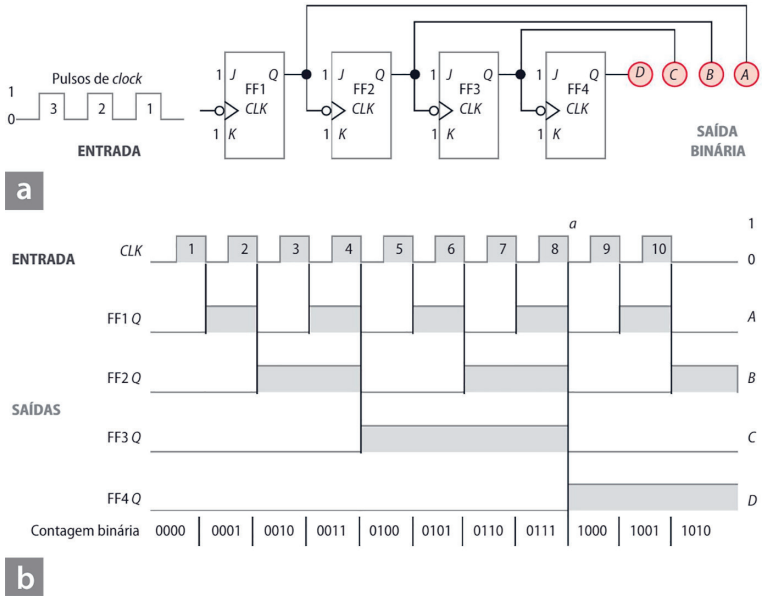


Figura 8. a) Contador assíncrono com FFs tipo JK; b) diagrama no tempo da resposta obtida em cada um dos bits (A, B, C e D) que compõem o número.

Fonte: Adaptada de Tokheim (2013, p. 233).

O contador pode funcionar no sentido decrescente apenas utilizando o pino da saída invertida $\overline{Q}$ para acionar o *clock* (>) em cada interligação de FF, em vez do pino da saída direta Q. Nesse caso, ainda podem ser utilizadas as entradas $\overline{PS}$ para iniciar o contador no valor máximo. Entretanto, perceba que, ao final da contagem, seja ela crescente ou decrescente, os FFs vão alternar novamente e retornarão ao valor inicial, dando sequência à contagem, conforme explica Tokheim (2013).

Contadores síncronos

Ao utilizar um contador de modo assíncrono, o último FF depende de que o anterior já tenha alternado sua saída, e o anterior depende do imediatamente

anterior a ele, e assim sucessivamente. Logo, haverá um acúmulo de atraso de propagação, já que nenhum circuito lógico opera de forma instantânea. Quanto maior for a quantidade de bits, maior será o erro.

Os contadores também podem funcionar no **modo síncrono** (Figura 9). Nesse caso, todos os FFs utilizam o mesmo sinal de entrada para o pino de *clock*. Mas, para que os FFs alternem de valor em momentos distintos, será necessário utilizar combinações lógicas a partir das saídas dos FFs que antecedem cada unidade binária, para servir de sinal para o pino T (ou J e K ao mesmo tempo, como no exemplo anterior). Dessa forma, o circuito aumenta de tamanho, mas problemas decorrentes do excesso de atraso de propagação serão reduzidos.

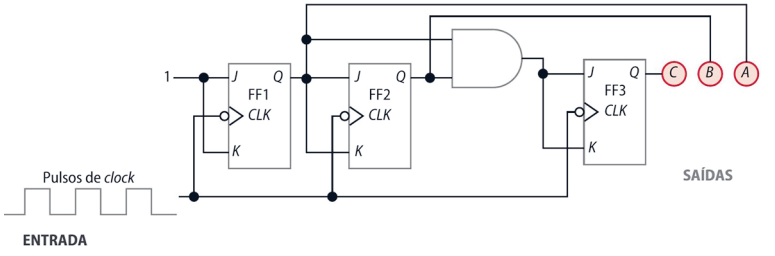


Figura 9. Contador síncrono de três bits (de zero a sete).
Fonte: Tokheim (2013, p. 235).

É possível, ainda, utilizar as entradas de PRESET e CLEAR para saltar estados em uma contagem, ou limitar o máximo e o mínimo de um contador. Um exemplo típico desse tipo de uso são contadores de 4 bits utilizados para contar até o limite de uma unidade decimal (BCD). São necessários os 4 bits para representar o número 9 em decimal, mas este não poderá reiniciar a contagem, somente no caso 1111_2. A solução é utilizar uma combinação de portas AND que, ao detectar que o primeiro e o último bits são verdadeiros, em 1001_2, o pino CLEAR será acionado. Assim, o contador vai retornar a zero no incremento seguinte, conforme leciona Tokheim (2013).

Máquinas de estados finitos

Os circuitos sequenciais podem ser utilizados para dividir as tarefas em etapas. Para identificar cada uma delas, é comum que se utilizem contadores em conjunto com codificadores para controlar os demais circuitos e o momento em que cada um será acionado.



Exemplo

Em um microprocessador, por exemplo, os barramentos de dados são compartilhados entre vários circuitos diferentes, a fim de economizar em ligações necessárias e, também, facilitar o projeto do circuito. É claro que essas funções não podem nem devem ser utilizadas simultaneamente, já que o resultado seria algo caótico, sem qualquer sentido para o que fora requisitado.

Para cada comando solicitado ao microprocessador, este deverá executar uma série de procedimentos a fim de concluir a operação integralmente. Pode ser ordenado, por exemplo, que seja efetuada uma soma entre dois números disponíveis na memória. Nesse caso, o processador terá de acessar o endereço de cada um deles individualmente e armazenar cada um deles em um registrador, e o conjunto desses registradores servirá de entrada para o circuito de soma. Ao final, o resultado do circuito somador deverá ser armazenado em outro endereço de memória, mas utilizando o mesmo barramento pelo qual entraram cada um dos dados dos registradores.

Somente um valor pode utilizar o barramento a cada momento; assim, um contador é iniciado no começo da operação. Cada passo (combinação binária) é decodificado em conjunto com o código da operação, para acionar os circuitos lógicos necessários naquele momento, bem como direcionar multiplexadores e demultiplexadores que permitam destinar o barramento de forma adequada a cada um dos circuitos.

O exemplo anterior é bastante complexo, mas facilita o entendimento de como vários dos circuitos vistos até aqui costumam ser utilizados em conjunto e fornece uma visão da necessidade de um contador atuar como uma **máquina de estados**. Porém, há exemplos mais simples para explicar a máquina de estados.

Pense em um semáforo com três LEDs. Há uma sequência rígida a ser seguida. Podem ser utilizados contadores para controlar o tempo de mudança dos semáforos, e é utilizado um contador para informar em qual situação o

semáforo se encontra, sendo incrementado para a posição seguinte, permitindo que um decodificador acione os LEDs corretos em cada momento. Esse processo também depende de uma máquina de estados.

O projeto desse tipo de circuito sequencial pode ser simplificado utilizando um diagrama, conhecido como **diagrama de estados**. Nele, são indicados todos os estados possíveis para uma máquina de estados e como devem ser os avanços entre cada uma das situações. Um diagrama que possui um ciclo único sem qualquer exceção pode ser representado como na Figura 10, em que os estados possuem uma única mudança, sempre na mesma direção e para o mesmo estado.

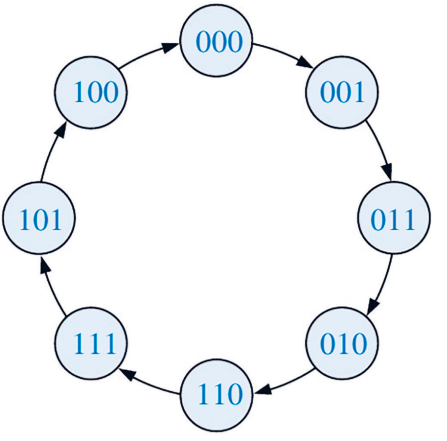


Figura 10. Diagrama de estados para um contador de código Gray.

Fonte: Floyd (2007, p. 464).

As setas indicam qual será o próximo estado a partir do atual. Não havendo qualquer outra indicação, é assumido que a transição ocorre a cada pulso de *clock* do sistema. Mas podem existir outras condições para a transição, como também duas ou mais condições diferentes que ditarão caminhos diferentes a partir de um mesmo estado. Nesse último caso, haveria mais de uma seta saindo de um estado em direção a outro, mas deveriam ser especificadas

as variáveis de condição e o estado necessário para que a transição ocorra. A Figura 11 representa um diagrama de estados de um contador de código Gray que, caso a entrada *Y* seja 0, vai funcionar no sentido decrescente, e caso seja 1, será crescente. Portanto, existem duas condições de transição para cada estado e sentidos diferentes.

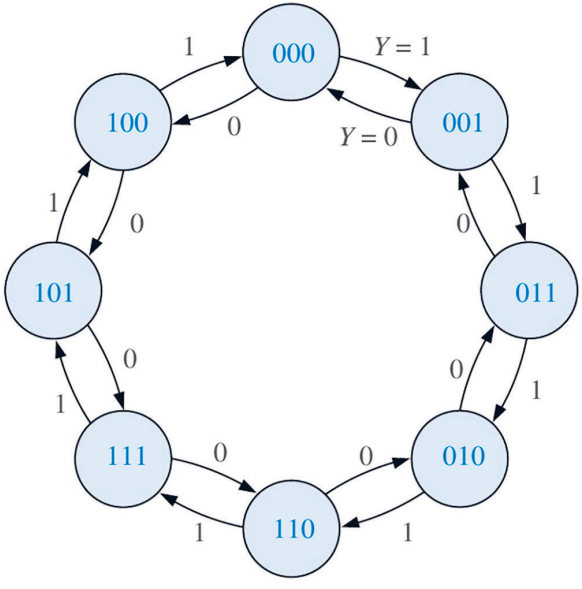


Figura 11. Diagrama de estados para um contador de código Gray que opera tanto em sentido crescente (*Y* = 1) quanto decrescente (*Y* = 0).
Fonte: Floyd (2007, p. 470).

Não é necessário que um estado salte para o imediatamente seguinte ou anterior. Poderão existir situações em que os estados mudam drasticamente para outras posições. Por exemplo, contadores utilizados em um relógio, em que dois contadores vão contar de 0 a 59, sendo utilizados para segundos e minutos, enquanto o outro contará de 0 até 23, para representar a hora. Mesmo para cada tipo de contador, haverá transições não lineares do contador, em estado igual a 9 para 0 novamente, típico de um código BCD.

Referências

FLOYD, T. L. *Sistemas digitais:* fundamentos e aplicações. 9. ed. Porto Alegre: Bookman, 2007.

TOCCI, R. J.; WIDMER, N. S.; MOSS, G. L. *Sistemas digitais:* princípios e aplicações. 11. ed. Rio de Janeiro: Pearson, 2011.

TOKHEIM, R. *Fundamentos de eletrônica digital:* sistemas combinacionais. 7. ed. Porto Alegre: AMGH, 2013. v. 1.

Leituras recomendadas

CAPUANO, F. G.; IDOETA, I. V. *Elementos de eletrônica digital.* 41. ed. São Paulo: Érica, 1997.

SZAJNBERG, M. *Eletrônica digital:* teoria, componentes e aplicações. Rio de Janeiro: LTC, 2014.

TOKHEIM, R. *Fundamentos de eletrônica digital:* sistemas sequenciais. 7. ed. Porto Alegre: AMGH, 2013. v. 2.

VAHID, F. *Sistemas digitais:* projeto, otimização e HDLs. Porto Alegre: Bookman, 2008.

Encerra aqui o trecho do livro disponibilizado para esta Unidade de Aprendizagem. Na Biblioteca Virtual da Instituição, você encontra a obra na íntegra.

Conteúdo:

SAGAH
SOLUÇÕES EDUCACIONAIS INTEGRADAS